# Pipelining Hazards

ANISHA M. LAL

# Hazards

- Hazards cause the pipeline to stall because of some conflict in the pipeline.

Prevents the next instruction to be executed.

Types:

1. Structural hazards- contention for same hardware resource
2. Data hazards- dependency between instructions
3. Control hazards- branch/jump instruction stall the pipeline until the target address is loaded in PC.

# Structural Hazard

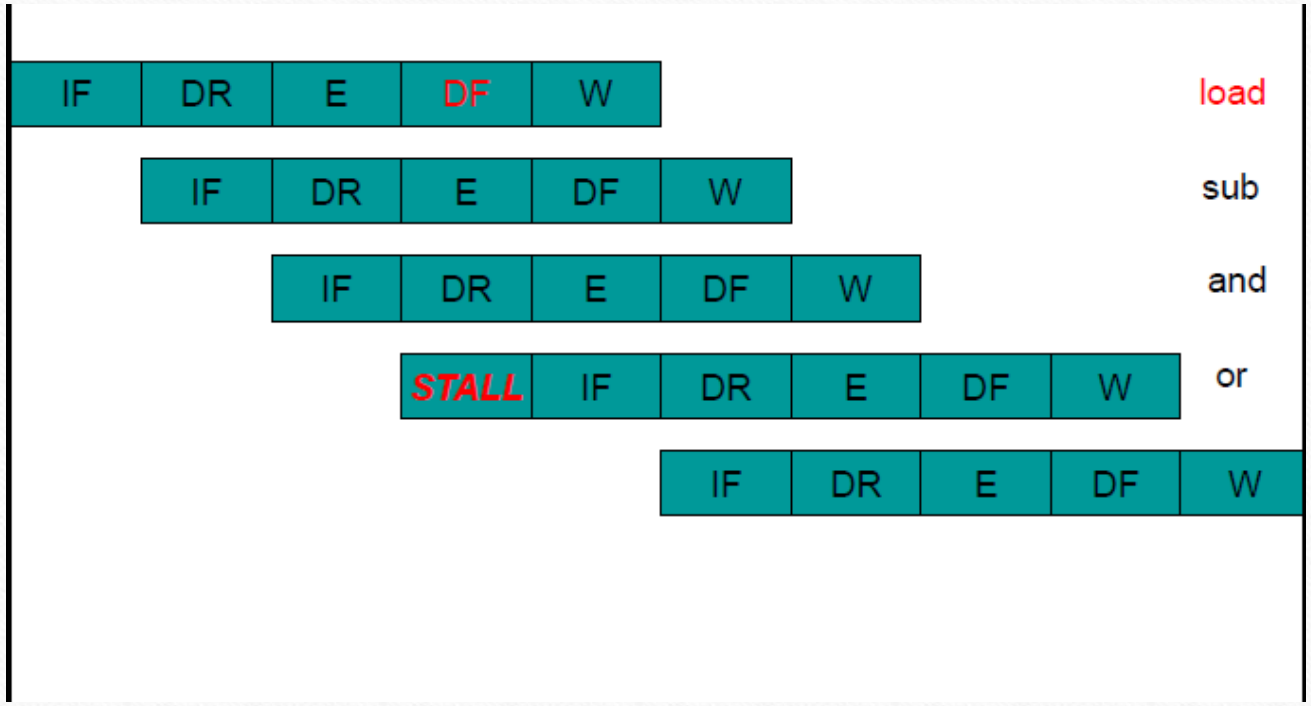
**Structural Hazards**

- Resource conflicts in the pipeline

Examples

- Single memory port shared for instruction and data access
- Register file without a separate write port

# Structural Hazard



# Structural Hazard

- IF and DF compete for single memory port

Ideal Machine

- No stalls, 1 cycle per instruction
- Assume 30% of instructions access data
- With structural hazard, 1.3 cycles per instruction
- Performance has gone down by 30%

Solutions:

- Pipeline stall (insert bubble)
- Have 2 memory ports for shared instruction-data cache-memory (expensive)
- Have separate instruction cache-memory and data cache-memory

# Data Hazard

- Data hazard – any condition in which either the source or the destination operands of an instruction are not available at the time expected in the pipeline. So some operation has to be delayed, and the pipeline stalls.
- Three Generic Data hazards:

  RAW- Read after Write.

  WAR – Write after read

  WAW- Write after write.

# RAW

- Instr1 followed by Instr2

  add r1, r3, r2

  add r4, r5, r1

Instr2 tries to read operand before Instr1writes it

- Can be due to true “data dependency” (data must be produced before it can be consumed)
- Or can be due to pipeline staging (data already produced, but not yet written to general register file

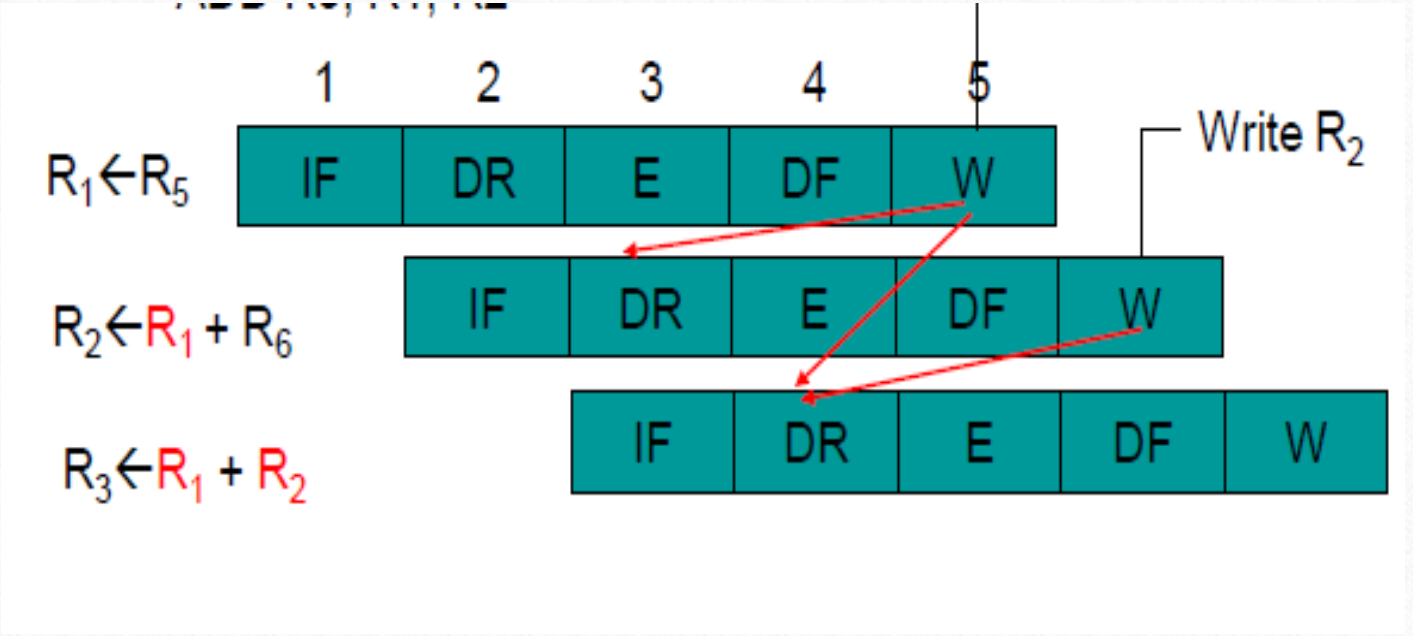
# RAW

- Overlapping instructions cause dependencies on data (RAW)

e.g., MOVA R1, R5

ADD R2, R1, R6

ADD R3, R1, R2



# WAR

- Instr1 followed by Instr2

  ld r1, (r3)+

  add r3, r4, r1
- Instr2 tries to write operand before Instr1 reads it
- Instr1 gets wrong operand
- Can't happen in the 5-stage pipeline we just covered
  - All instruction take 5 stages
  - Reads are always in stage 2
  - Writes are always in stage 5

# WAW

- Instr1 followed by Instr2

  mul r1, (r0), (r2)

  add r1, r5, r6

- Instr2 tries to write operand before Instr1 writes it
- Leaves wrong result (Instr1, not Instr2)
- Can't happen in our 5-stage pipeline because
  - All instructions take 5 stages
  - Writes are always in stage 5

# Data Hazards - Solutions

- Software – Compiler
- Hardware- Hardware stalls and Hardware Data Forwarding.

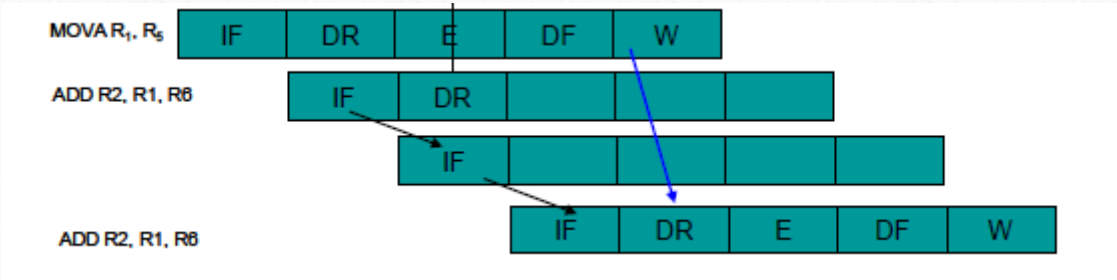
Software:

- Software delay (compiler or machine code programming to insert NOPs)

```
MOVA R1, R5
NOP
NOP
ADD R2, R1, R6
NOP
NOP
ADD R3, R1, R2
```

# Hardware solution

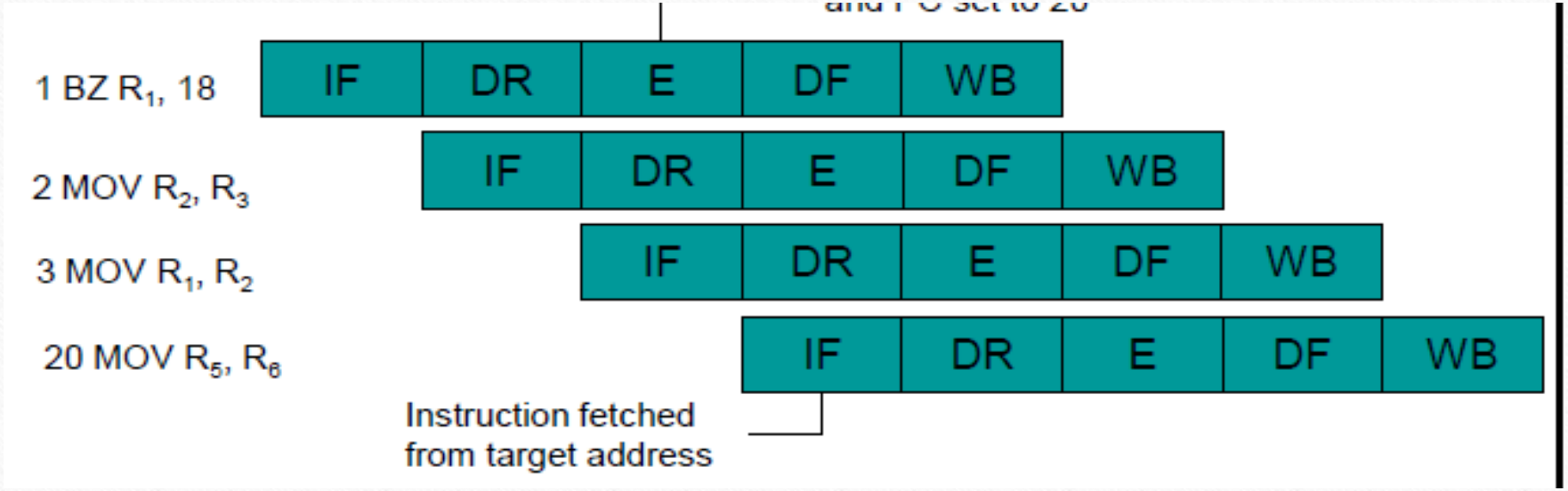
- Hardware – stalls



# Control Hazard

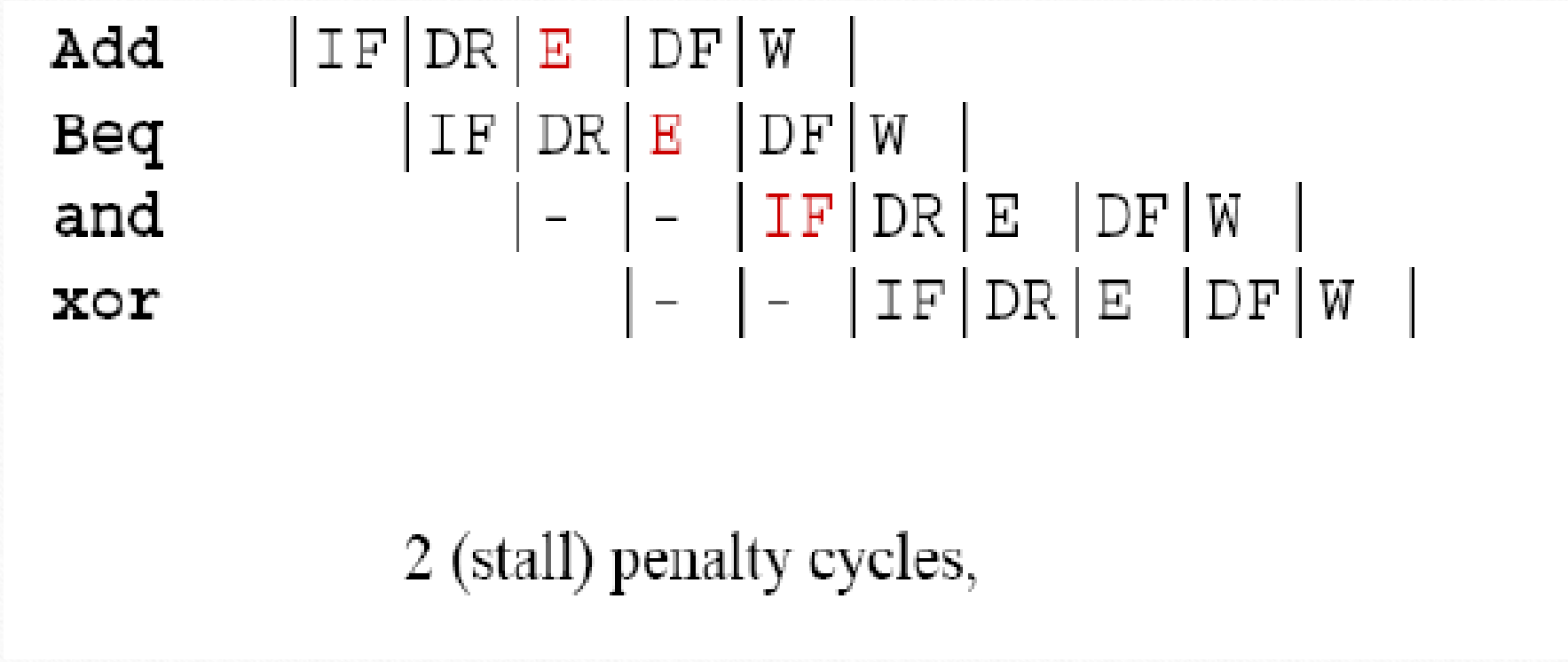
- For branch or jump instruction, the correct instruction to execute is not known in time (at the start of the IF stage of the instruction after the Branch)
- Condition not yet determined (for conditional branch instruction)
- Target instruction address not yet calculated

- Conflicts that arise from changes in the Program Counter
- Branch instructions
- ISA may allow code to run useful instructions
- Can use branch prediction to improve performance



# Solution 1: stall

The instructions after the branch are stalled, until the branch condition is checked and target address is generated



# Solution 2

Perform target address calculation earlier in DR stage

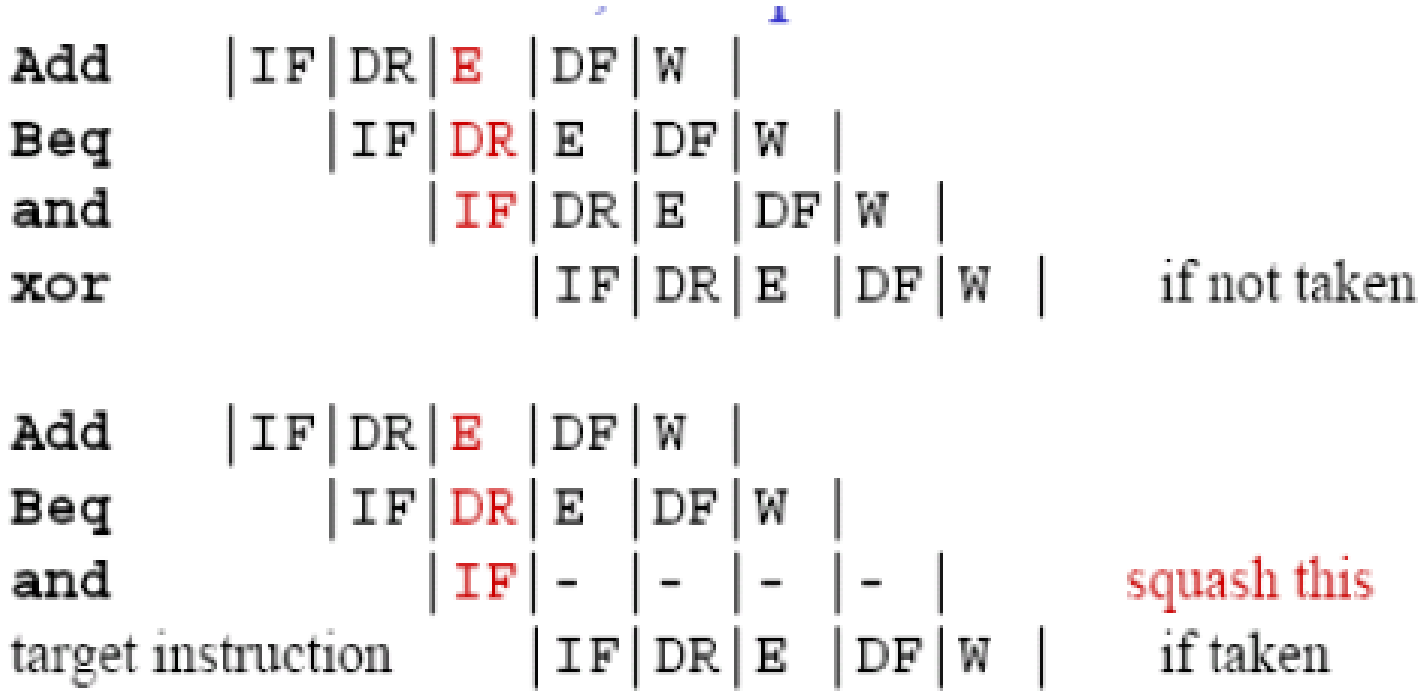
```
Add   |IF|DR|E |DF|W |
Beq      |IF|DR|E |DF|W |
and         |- |IF|DR|E |DF|W |
xor            |- |IF|DR|E |DF|W |
```

Reduced penalty cycles from 2 to 1,

Need a separate branch adder in the DR stage to calculate PC+displacement

# Solution 3a

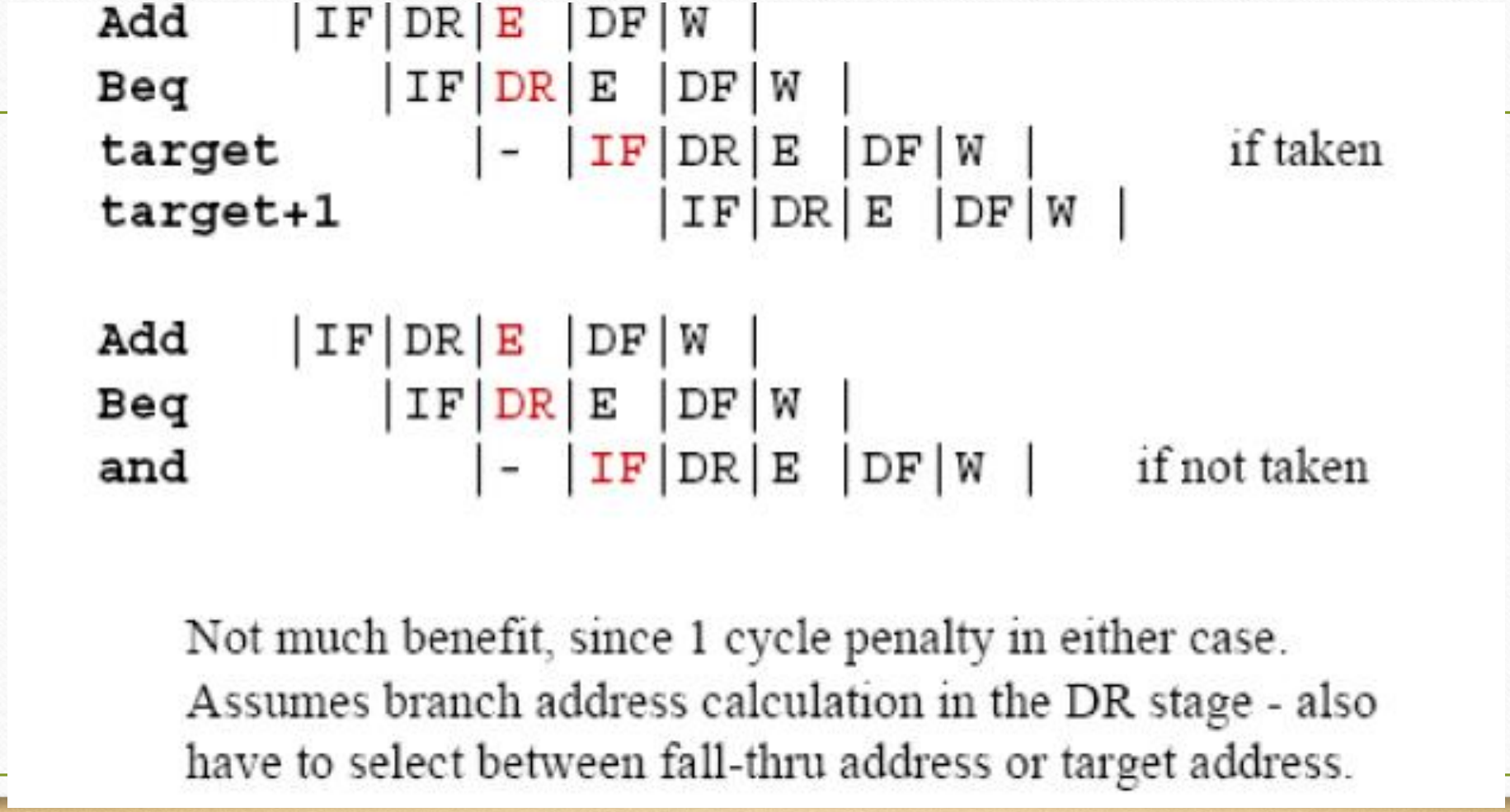
Assume branch not taken, fixup if taken




If branch is not taken (as predicted), then no penalty
else 1 penalty cycle.
Assumes branch address calculation in the DR stage

# Solution 3b

Assume branch taken, fixup if not taken




Not much benefit, since 1 cycle penalty in either case.
Assumes branch address calculation in the DR stage - also have to select between fall-thru address or target address.

# Solution 4

Delayed branch (ISA change)

```
Add                |IF|DR|E |DF|W |
Beq                   |IF|DR|E |DF|W |
(delay slot)             |IF|DR|E |DF|W |
fallthru or target          |IF|DR|E |DF|W |
```

ISA specifies that the branch, if taken, only takes place after a delay of x instructions. (Above, x=1)

Branch adder in DR stage to calculate PC+displ, or PC+4+displ (depending on ISA definition)

# Solutions to Control Hazard

- Micro-architecture solution
  - Stall the pipes
  - Calculate target address and condition earlier in pipeline (DR)
  - Assume branch always goes untaken (taken) and fix up pipe if it is actually taken (untaken)

- ISA solution

  Have delay branches

  Branch target takes place after n (delay) instructions

  For n penalty cycles, must have (n-1) stages between IF and the stage where target and condition are determined

  Have instruction which separate target address calculation and condition generation from actual branching, so these can be executed earlier (e.g., IA-64)

- Branch prediction

# Handling Hazard

- Avoid some hazards "by design"
- Eliminate DF-IF structural hazard by having separate I-cache and D-cache
- Eliminate WAR by always fetching operands earlier in pipe (DR)
- Eliminate WAW by doing all Ws in order (last stage, static) – not always the best micro-architecture decisions though!
- Delayed branch in ISA to reduce control hazard penalty
- Detect and resolve remaining ones
- Stall or forward (if possible)